

NexusFlow Analytics: Master Systems Architecture & SOP Canvas

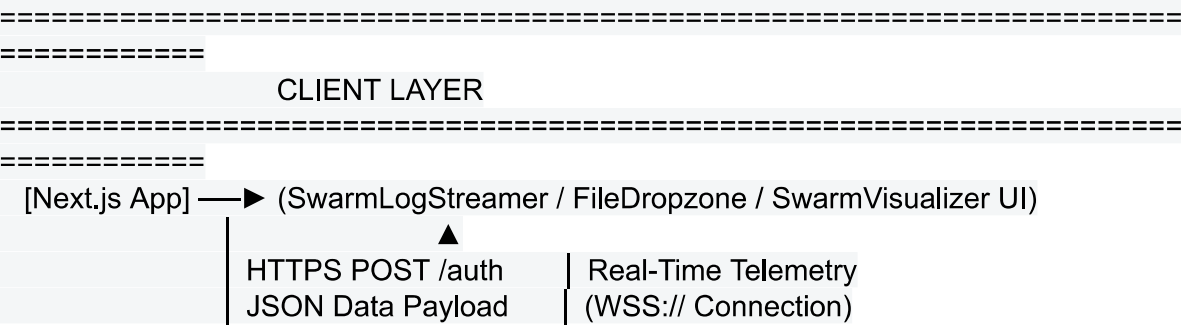
Document Type: Technical Architecture Record & Operations Manual
Prepared By: SOP Manager Agent
Repository Access Reference: LGuition/nexusflow-portal
System Integrity Status: Enforced

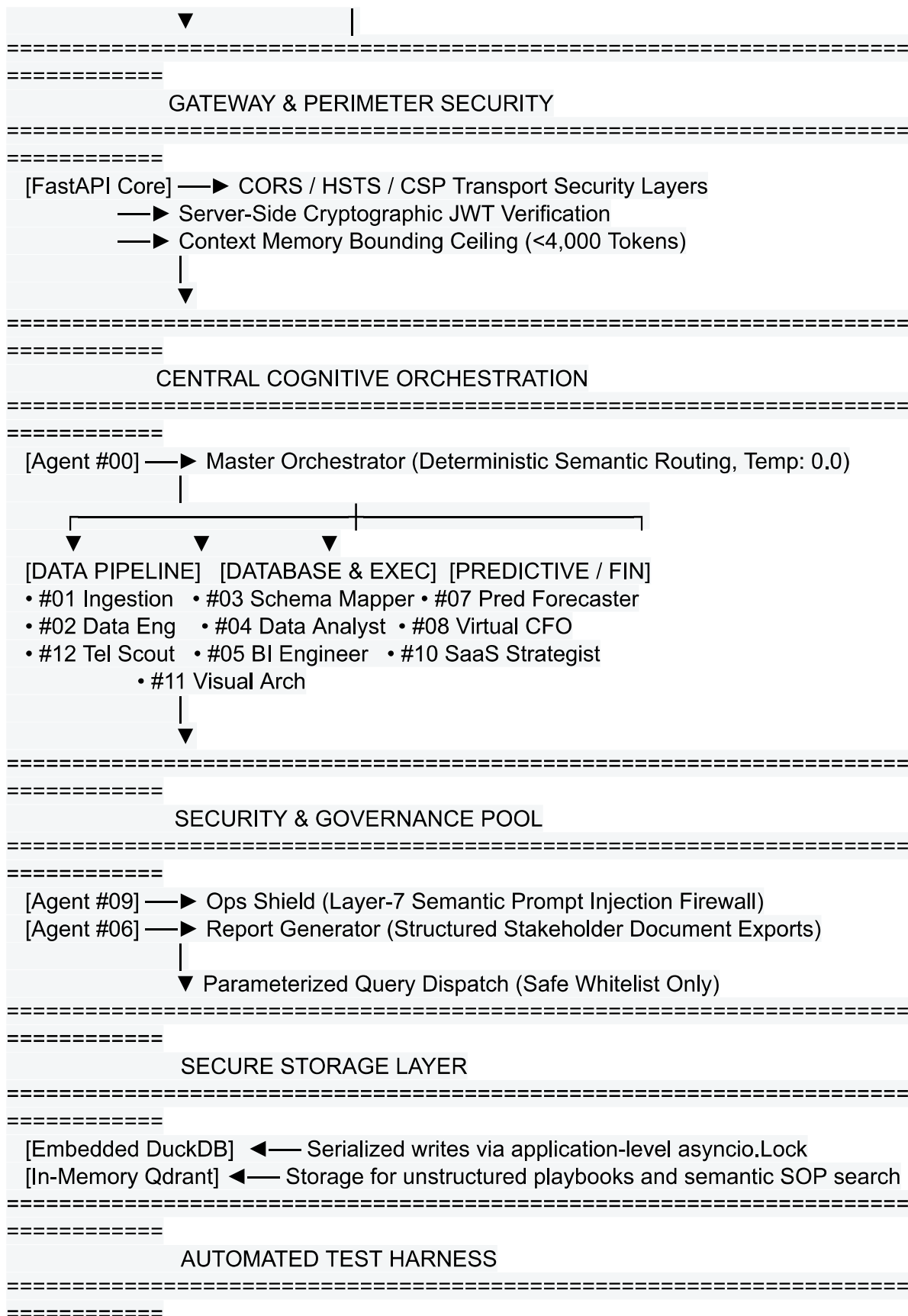
1. Executive Summary & Core Engineering Rules

NexusFlow Analytics is a full-stack, multi-tenant AI Business Intelligence platform connecting an autonomous multi-agent microservice swarm to a client's operational data layer. To maintain system stability, the platform implements these mandatory development standards:

- **Continuous Evolutionary Prototyping (CEP):** We build complete, vertically sliced features from top to bottom (Ingress -> Database -> Visualizer) without using unexecutable boilerplate or code placeholders.
- **Harness-Driven Development (HDD):** Write strict programmatic assertions inside the test suite (backend/swarm_test.py) to prevent model hallucinations and runtime regressions.
- **The Single-Writer Principle:** DuckDB is a single-writer engine. All concurrent database write operations must be serialized via an application-level mutex lock (asyncio.Lock).
- **Server-Side Boundary Enforcement:** Trust-based client-ID tracking strings are entirely removed from client-facing URLs. Every data path must authenticate via server-side JSON Web Token (JWT) signature verification.

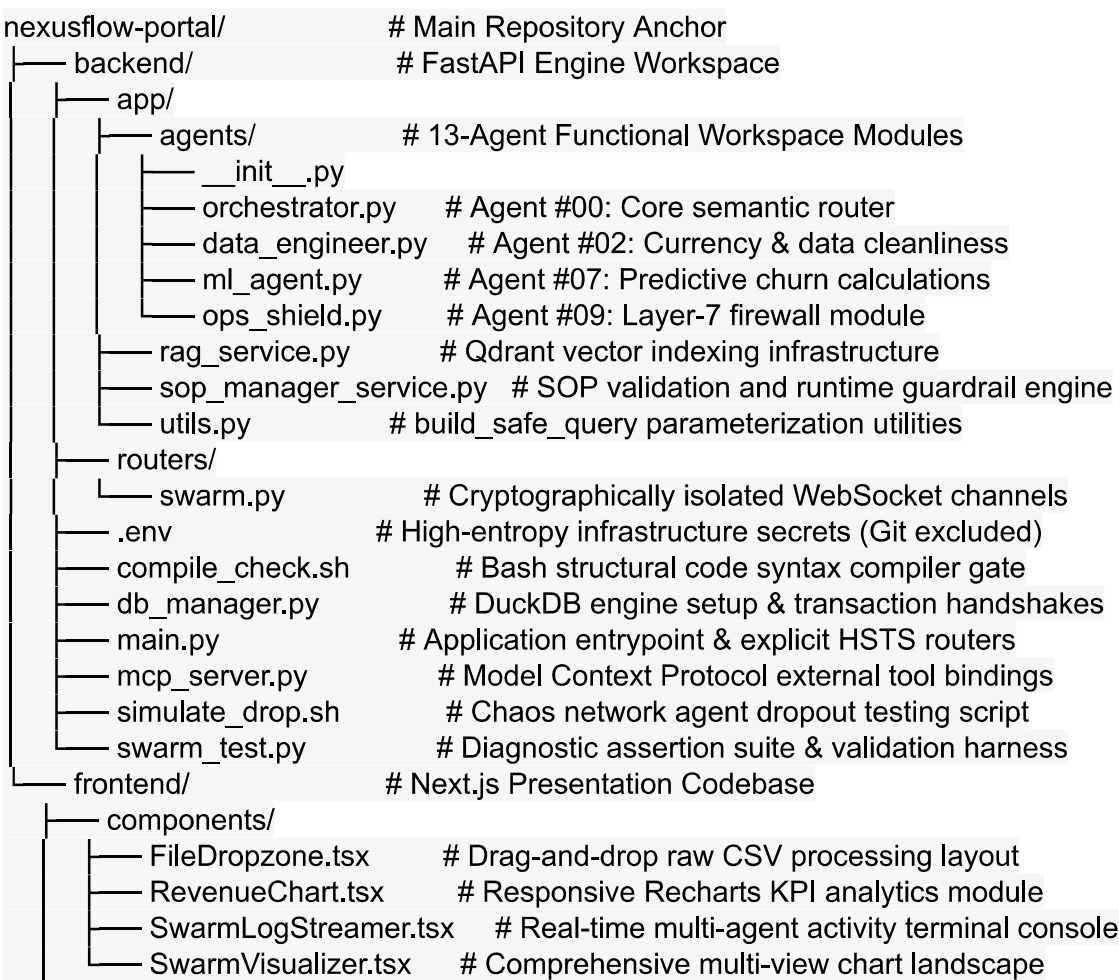
2. Platform Architecture Blueprint





• swarm_test.py • compile_check.sh • simulate_drop.sh • checkpoint.sh
=====

3. Production Monorepo Directory Layout



Project Directory File Tree Diagram

This visual folder directory map outlines exactly where all 10 background sub-agent scripts, core

API endpoints, pipeline components, and frontend Next.js modules live within your enterprise repository layout:

```
nexusflow-analytics/
├── backend/
│   ├── main.py                    # Master Supervisor
Registry, Token-Capping, BYOK Handler, Async Loops & FastAPI
Gateway
│   ├── db_manager.py             # Phase 1: Embedded DuckDB
Columnar Engine, Safe SQL Parameter Builder & Lineage Logs
│   ├── mcp_server.py             # Phase O-1: Model Context
Protocol Standardized Tool Server Registry
│   ├── swarm_test.py             # Phase 6: Automated Testing
Suite & Hallucination Guardrail Harness
│   ├── simulate_drop.sh          # Phase G: Chaos Engineering
Agent Dropout Simulator Script
│   ├── compile_check.sh          # Phase H: Bash Code
Integrity & Syntax Validator Gate Script
│   ├── requirements.txt          # Project Dependencies
(fastapi, uvicorn, duckdb, pydantic, httpx, openai, pandas)
│   ├── Dockerfile                # Base Container
Virtualization Build Script
│   ├── environment.env           # Local Environment
Variables & Secure API Keys
│   └── app/
│       ├── main.py               # Legacy FastAPI routing
setup (preserved for endpoint maps)
│       ├── ml_agent.py           # Phase 3: Scikit-Learn
Predictive Churn Analytics Worker
│       └── rag_service.py        # Phase 3: In-Memory Qdrant
Vector Search Service Engine
│   └── agents/
│       ├── __init__.py
│       ├── ai_architect.py       # Agent #01: AI Engineering
Expert (LLM Orchestration Parameters)
│       ├── code_customizer.py    # Agent #02: Software
Engineer AI Agent (Dynamic Features Generation)
│       ├── data_engineer.py      # Agent #03: Data Engineer
Agent (Consolidated Self-Healing Ingestion + ETL)
│       ├── neural_engine.py      # Agent #04: Predictive
Forecaster / Deep Learning Agent (Time-Series Math)
│       ├── self_optimizer.py     # Agent #05: Machine
Learning AI Agent (Model Drift Retraining Loops)
│       ├── math_analyst.py       # Agent #06: Data Science
Agent (Predictive Models & Regression Calculations)
│       └── storyteller.py        # Agent #07: Data Analyst
```

```

Agent (Natural Language to SQL Generation)
|   |   |— virtual_cfo.py           # Agent #08: BI Analyst /
Virtual CFO Briefing Synthesizer (Text Briefs)
|   |   |— ops_shield.py           # Agent #09: Operational
Environment Shield (Consolidated 6-Guard Platform Roles)
|   |   |— data/
|   |       |— raw/                 # Temporary Landing Sandbox
for Uploaded Client CSV Files
|   |       |— secure/              # Isolated, Multi-Tenant
Encrypted Database Data Storage & Lineage Audits
|   |       |— nexusflow.duckdb     # Production Embedded
Columnar Data Ledger Warehouse
|   |   |— frontend/
|   |       |— package.json         # Node.js Layout Dependency
Packages
|   |       |— tailwind.config.js   # Utility-First Tailwind CSS
Parameter Configurations
|   |       |— src/
|   |           |— app/
|   |               |— layout.js     # Global Application Shell &
High-Hierarchy Navigation Layout
|   |               |— page.js       # Core Responsive UI Landing
Dashboard Screen
|   |               |— page.tsx      # Production Dark-Themed
Responsive Control Plane Web Shell
|   |               |— components/
|   |                   |— ClientContext.tsx # Global State
Management for Multi-Tenant ID Filtering
|   |                   |— SearchBar.jsx    # Universal Cognitive
Search Input Interface
|   |                   |— DropZone.jsx     # Interactive
Drag-and-Drop CSV Processing Upload Zone
|   |                   |— FileDropzone.tsx # Phase 4: Secure
Drag-and-Drop Ingestion Upload Interface
|   |                   |— FileUpload.tsx   # Phase 4: Machine
Learning Target Predictive File Trigger
|   |                   |— CognitiveSearchbar.tsx # Phase 5: Universal
Natural Language Bar & Swarm Trace UI
|   |                   |— SwarmVisualizer.tsx # Phase 5: Recharts View
Engine, Table & SQL Inspector Tab
|   |                   |— RevenueChart.tsx  # Phase 5: Recharts
Executive Grid Metrics Engine Component
|   |                   |— SwarmLogStreamer.tsx # Real-Time Telemetry
Terminal & WebSocket Lifecycle Monitor

```

4. Finalized 13-Agent Operational Workforce Matrix

Agent ID	Role Descriptor	Operational Domain	Functional Responsibility
#00	Orchestrator	Semantic Routing	Determines which specialist agent is best suited to process an inbound prompt.
#01	Ingestion Engine	Data Pipeline	Parses raw incoming user CSV files and webhooks.
#02	Data Engineer	Data Pipeline	Audits schemas, normalizes fields, and handles data hygiene tasks.
#03	Schema Mapper	Database	Optimizes DuckDB table indexing and ensures multi-tenant column isolation.
#04	Data Analyst	Execution	Translates natural language requests into safe, whitelisted SQL strings.
#05	BI Engineer	Analytics	Tracks real-time SaaS subscription metrics, data variance, and platform KPIs.
#06	Report Generator	Exports	Synthesizes processed analytics into structured stakeholder-facing

Agent ID	Role Descriptor	Operational Domain	Functional Responsibility
			files.
#07	Predictive Forecaster	Predictive	Runs ARIMA modeling sequences to project 3-month MRR growth loops.
#08	Virtual CFO	Finance	Calculates live gross margins, monthly burn rates, and financial runways.
#09	Ops Shield	Cybersecurity	Layer-7 semantic firewall blocking injection and privilege escalation.
#10	SaaS Strategist	Strategy	Models margin expansion trajectories, pricing tier elasticity, and indexes competitor performance.
#11	BI Visual Architect	Analytics	Generates dynamic JSON map configurations for chart components.
#12	Telemetry Scout	Ingestion	Maps nested JSON streaming payloads from third-party APIs to flat storage templates.

5. SOP-001: Operational Ingress Routing & Firewall Policy

- **Target Agent Core Component:** Agent #00 Orchestrator (Temperature: 0.0).
- **Enforcement Trigger:** Any incoming user prompt via the authenticated WebSocket upgrade route (routers/swarm.py).

Phase 1: Security Audit Interception

1. The Orchestrator **MUST** intercept all raw input text strings and immediately pass the data segment to **Agent #09 (Ops Shield)** prior to invoking any parsing logic.
2. The payload string **MUST** be programmatically audited for prompt injection signatures, adversarial role overrides, or back-door configuration alteration requests.
3. If a semantic violation is encountered, the system **MUST** transition into a fail-closed sequence: abort transaction execution immediately, output an alert to the immutable lineage audit tracker, and drop the WebSocket pipeline frame.

Phase 2: Token Budget & Context Verification

1. The engine **MUST** programmatically extract and verify the client_id parameter directly from the server-validated JWT signature scope.
2. The Orchestrator **MUST** truncate or buffer the request transaction array window to freeze the total token consumption footprint strictly beneath a **4,000 token ceiling**.

Phase 3: Whitelisted Execution Dispatch

1. The incoming user intention **MUST** map explicitly to one of the isolated functional workforce agent pools. Arbitrary or unmapped request intents **MUST** be dropped cleanly with a non-technical error code returned to the frontend.
2. If processing an analytics lookup via **Agent #04 (Data Analyst)**, the Orchestrator **MUST** channel the runtime context through the build_safe_query parameters.
3. The generated SQL string **MUST** be checked against the strict column whitelist. Raw, unparameterized strings output from an LLM model **MUST NOT** execute on the data layer under any condition.

Next Action Steps for the Team

- **Gemini (Code Builder):** Can immediately copy this structural markdown file to build out the functional code wrappers in backend/app/sop_manager_service.py or map the API hooks in backend/main.py.
- **Claude (Inspector):** Use this system blueprint to run adversarial isolation checking, test the concurrency limits of the asyncio.Lock in backend/db_manager.py, and audit the security rules defined in your test suite.

What is the next protocol you would like to design for your workspace? I can map out the technical implementation scripts for **backend/routers/swarm.py** to stand up the authenticated

WebSockets, or write the safe database insertion queries inside **backend/db_manager.py**.